

EMBEDDED CAREER TRANSITION READINESS CHECKLIST

Permanent bridge-job preparation: VDC → Hardware / Test / R&D → Embedded Systems

Print this document. Check items only when you can demonstrate them without relying on a step-by-step tutorial.

Purpose: This is NOT a list of everything an embedded systems engineer must know. It is the pre-transition readiness package for becoming credible and competitive for a PERMANENT, FULL-TIME bridge job such as R&D/engineering lab technician, electronics/product test technician, hardware test/validation technician, hardware integration/test technician, or embedded-test/HIL technician. The checklist must NOT make you wait until you are already engineer-level before applying.

MUST: Complete before the active application campaign. **SHOULD:** Strongly preferred; can be finished while applications are already underway. **STRETCH:** Differentiator for stronger roles; do not delay applications solely for this.

Certification intentionally excluded from this checklist.

CURRENT COURSE CONTEXT - v15.8

This checklist concerns practical career evidence; CETa exam preparation is handled separately by the booklet, competencies and course. Mark only work you can demonstrate. Virtual and physical evidence remain distinct. Website evidence indices do not establish job qualification; use real projects, measurements and reports.

How to Use This Checklist

The objective is verified bridge-job competence, not 100% mastery. Complete the COMMON CORE, then complete ONE target-role add-on. You do not need to satisfy every role family before applying. Skills marked SHOULD/STRETCH can continue while interviews are already underway. The final scorecard controls the application trigger; the detailed sections are a learning/proof inventory, not a requirement that every single box be perfect.

The active-application green light

Check	Gate	Green-light standard
[]	1. Electronics core	Core electronics/schematic/datasheet items are solid enough to work safely around low-voltage electronic hardware: DC circuits, components, digital basics, power, sensors, ESD, and basic measurement.
[]	2. Programming + automation core	You can write and explain basic C for an MCU project and use Python for serial communication, logging, simple automation, and test data. Deeper low-level C items are role-track depth, not universal blockers.
[]	3. MCU + interface proof	You have one working ARM/STM32-class MCU project that demonstrates GPIO, timer/interrupt behavior, UART, and at least ONE board-level serial bus (I2C or SPI). Both I2C and SPI plus CAN are required only when the target role benefits from them.
[]	4. Lab + fabrication competence	You can safely use a DMM, bench supply, oscilloscope, soldering tools, and basic ESD controls. Project 1 includes basic logic-analyzer and SWD/JTAG proof; deeper instrument/debugger proficiency depends on the target role.
[]	5. Troubleshooting evidence	You can explain at least 3 real hardware/firmware/test failures you diagnosed using measurements and a structured isolate-test-confirm process; at least one should involve a physical/electrical problem.

Check	Gate	Green-light standard
[]	6. Project + role-track evidence	Project 1 is complete and interview-ready. In addition, ONE target-role add-on in Section 15 is substantially complete. Project 2 is required only for Hardware Validation / Embedded Test / HIL targeting; other tracks may substitute a relevant prototype, rework, harness, bring-up, or troubleshooting proof artifact.
[]	7. Professional proof	Targeted resume, GitHub/portfolio, project photos/evidence, and 5+ technical/behavioral stories are ready. You can demonstrate skills without pretending personal projects are professional experience.
[]	8. Offer/exit filter - NOT an application blocker	Before leaving your current job, the actual offer must be permanent/full-time, financially acceptable, school-compatible, and genuinely closer to electronics/hardware/R&D work. This gate is checked at OFFER time, not before applications begin.

How to decide whether a box is truly complete

- [] You can explain the concept without reading notes.
- [] You can point to code, measurements, a schematic, or a project artifact that proves the skill when the item is hands-on.
- [] You can answer at least one "what went wrong and how did you debug it?" question about the skill.
- [] You understand the limits of your knowledge and can state what you would check next instead of bluffing.
- [] For MUST items, a tutorial that you copied line-for-line does NOT count as proof.

Contents

1. Electrical & Electronics Fundamentals
 2. Digital Electronics, Power, Sensors & Schematics
 3. C Programming for Embedded Work
 4. Python for Hardware Test & Validation
 5. Microcontrollers & Embedded Architecture
 6. Communication Protocols
 7. Lab Equipment & Hands-on Electronics
 8. Troubleshooting & Root-Cause Analysis
 9. Git, Linux & Development Workflow
 10. Engineering Documentation & Test Discipline
 11. Required Project 1 - STM32 Telemetry/Control System
 12. Role-Track Project 2 - Automated Hardware Validation / HIL (Required only for validation/HIL targeting)
 13. Strong Differentiator Project - Custom PCB
 14. Interview Readiness
 15. Role-Specific Add-On Checklists
 16. Resume / Portfolio / Application Package
 17. Final Green-Light Scorecard
- Appendix - Market Signals Used to Build This Checklist

1. Electrical & Electronics Fundamentals

Target standard: technician/test-level fluency, not upper-division EE theory. You must be able to reason through simple circuits, select basic components, make measurements, and explain why a circuit behaves the way it does.

DC circuit fundamentals

Check	Skill / concept	What you must be able to prove or explain
[]	Voltage, current, resistance	Explain each quantity, units, polarity/current direction conventions, and how they relate physically.
[]	Ohm's law	Calculate V, I, or R; choose a resistor for a target current; sanity-check measured values.
[]	Electrical power	Use $P=VI$, $P=I^2R$, $P=V^2/R$; estimate resistor power rating and device power draw.
[]	Series circuits	Calculate total resistance, current, voltage drops, and recognize an open circuit.
[]	Parallel circuits	Calculate equivalent resistance/current branches and recognize a short or unintended low-resistance path.
[]	Kirchhoff's Voltage Law	Reason around a loop and verify that measured voltage drops make sense.
[]	Kirchhoff's Current Law	Reason about current entering/leaving a node and simple branch currents.
[]	Voltage dividers	Calculate output voltage, explain loading error, and use dividers for sensing when appropriate.
[]	Continuity and resistance measurement	Know when power MUST be off; distinguish continuity from a meaningful resistance measurement.
[]	Open vs short circuit	Predict likely symptoms and the first measurements you would make for each.
[]	SI prefixes / engineering notation	Comfortably convert between G/M/k/base/m/u/n/p where relevant; read values such as 4.7 kOhm, 100 nF, 2.2 uF, 25 mA, and avoid 1,000x unit mistakes.

Passive components

Check	Skill / concept	What you must be able to prove or explain
[]	Resistors	Read common values/markings; understand tolerance, power rating, pull-ups/pull-downs, current limiting.
[]	Capacitors	Explain capacitance, charge/discharge, polarity, basic RC behavior, decoupling/bypass use, bulk capacitance.
[]	Inductors	Explain stored magnetic energy, opposition to current change, and basic use in filters/power conversion.
[]	RC time constant	Explain $\tau=RC$ and observe a charging/discharging waveform on an oscilloscope.
[]	Basic filters	Distinguish low-pass/high-pass behavior and recognize when filtering a noisy sensor or supply may help.
[]	Component ratings	Check voltage/current/power/temperature ratings in a datasheet before substituting parts.

Check	Skill / concept	What you must be able to prove or explain
[]	Tolerance and worst-case thinking	Explain why nominal values are not exact and how tolerance can affect thresholds or measurements.

Semiconductor fundamentals

Check	Skill / concept	What you must be able to prove or explain
[]	Diodes	Explain one-way conduction, forward voltage, polarity, reverse-bias concept, and flyback protection use.
[]	LEDs	Identify polarity and calculate a current-limiting resistor.
[]	Zener/TVS concept	Explain voltage clamping/protection at a basic level.
[]	BJT transistor	Explain transistor-as-switch concept, base current, load switching, and why a resistor is used.
[]	MOSFET	Explain gate/source/drain, N-channel low-side switching, V_{gs} , logic-level requirement, and flyback diode with inductive loads.
[]	Comparator	Explain threshold comparison and why hysteresis may be useful.
[]	Op-amp basics	Explain ideal high-level behavior, buffering/amplification concept, rail limitations, and common sensor-conditioning role.
[]	Datasheet pinout	Find pin functions, absolute maximum ratings, recommended operating conditions, and typical application circuit.

2. Digital Electronics, Power, Sensors & Schematics

Digital fundamentals

Check	Skill / concept	What you must be able to prove or explain
☐	Binary and hexadecimal	Convert common values; explain why hex is convenient for registers and bit masks.
☐	Logic 0 / logic 1	Understand logic-level thresholds concept; never assume every device uses the same voltage.
☐	AND, OR, NOT, XOR	Explain truth behavior and read a simple digital logic diagram.
☐	Pull-up / pull-down resistors	Explain floating inputs; choose/use pull-up or pull-down for buttons or buses.
☐	Open-drain/open-collector	Explain why an external pull-up may be required, especially on I2C.
☐	Debouncing	Explain mechanical switch bounce and implement hardware or software debouncing.
☐	Level shifting	Recognize when 5 V and 3.3 V devices cannot be connected directly; explain basic level-shifter need.
☐	PWM	Explain duty cycle/frequency and use PWM for LED brightness, motor control, or actuator command.
☐	ADC	Explain resolution, reference voltage, quantization, input range, and convert raw counts to engineering units.
☐	DAC concept	Explain converting a digital value into an analog level, even if your MCU project uses PWM instead.
☐	Clock / timing	Explain clock frequency, period, oscillator concept, and why timing mismatch causes interface failures.

Power and protection

Check	Skill / concept	What you must be able to prove or explain
☐	3.3 V / 5 V rails	Recognize common MCU voltage domains and measure rails before debugging software.
☐	Linear regulator / LDO	Explain what it does, dropout concept, heat/power dissipation, and input/output capacitor role.
☐	Buck converter concept	Explain why switching regulation is used for efficient step-down power.
☐	Current limiting	Use a bench supply current limit to protect a new circuit during bring-up.
☐	Reverse polarity / overvoltage concept	Recognize common protection methods and why they matter.
☐	Decoupling capacitors	Explain why 0.1 μ F-class local decoupling is placed near IC supply pins and what happens when supply noise is excessive.
☐	Ground reference	Explain that voltage is relative; identify common ground problems and avoid accidental ground loops in simple bench setups.
☐	ESD awareness	Use an ESD-safe work area and explain why MOS/CMOS devices are vulnerable.

Sensors, actuators, and interfacing

Check	Skill / concept	What you must be able to prove or explain
[]	Digital sensors	Read a sensor datasheet and identify power, interface, address/pins, timing, and conversion units.
[]	Analog sensors	Measure an analog output, scale it, and connect it safely to an ADC.
[]	Buttons/switches	Wire with pull-up/pull-down, debounce, and detect state changes.
[]	LED/buzzer output	Drive simple outputs within GPIO current limits or use a transistor when required.
[]	DC motor / relay concept	Explain why MCU GPIO cannot directly drive higher-current/inductive loads; use driver/MOSFET and protection diode.
[]	Servo concept	Generate an appropriate control signal and understand power separation concerns.
[]	Sensor calibration concept	Compare raw measurements against a reference and document offset/scale error.
[]	Fault states	Define what the system should do if a sensor disconnects, reading is impossible, or communication times out.

Schematics and datasheets

Check	Skill / concept	What you must be able to prove or explain
[]	Schematic symbols	Recognize resistor, capacitor, diode, LED, transistor/MOSFET, IC, connector, ground, power, switch.
[]	Net names	Trace a signal by net label rather than assuming all connections are drawn as continuous wires.
[]	Power tree	Trace power from input through protection/regulation to MCU and peripherals.
[]	Signal tracing	Follow one MCU pin through connector/components to the external device.
[]	Connector pinout	Verify pin numbering/orientation before connecting hardware.
[]	Datasheet electrical limits	Locate absolute maximum vs recommended operating conditions and explain why they are different.
[]	Timing diagram	Read basic UART/I2C/SPI timing diagrams and compare them to logic-analyzer captures.
[]	Register map	Find a peripheral configuration register, identify bits, and relate it to firmware settings.
[]	Reference manual vs datasheet	Explain that the MCU datasheet covers device/package/electrical facts while the reference manual describes peripheral behavior/registers.
[]	Errata awareness	Know to check device errata when behavior contradicts the documentation.

3. C Programming for Embedded Work - Core + Role-Track Depth

The standard here is not "software engineer interview LeetCode." It is: can you write reliable low-level C, understand memory and bits, read existing firmware, and debug the kinds of mistakes that break hardware control?

C language essentials - CORE MUST at a practical level

Check	Skill / concept	What you must be able to prove or explain
[]	Build/compile/link concept	Explain source → compiler → object files → linker → executable/firmware image at a high level.
[]	Types and integer widths	Use uint8_t/uint16_t/uint32_t/int32_t and explain why fixed-width types matter in firmware.
[]	Signed vs unsigned	Explain common comparison/overflow pitfalls.
[]	Variables and scope	Use local/global/static variables intentionally; explain lifetime and visibility.
[]	Functions	Write prototypes, definitions, parameters, return values, and decompose firmware into modules.
[]	Header/source files	Create .h/.c modules and use include guards or #pragma once.
[]	Arrays	Use fixed arrays safely and explain index bounds.
[]	Strings	Understand null termination and avoid obvious buffer-overflow mistakes.
[]	Structs	Group related state/configuration data into a struct.
[]	Enums	Use enums for states, errors, or modes instead of magic numbers.
[]	Macros	Use simple constants/macros carefully; understand why macros can be dangerous with side effects.
[]	const	Protect read-only data and understand pointer-to-const concept at a practical level.
[]	volatile	Explain why memory-mapped registers and variables changed by ISRs/hardware may require volatile; also explain that volatile is NOT a thread-safety primitive.
[]	Pointers	Take addresses, dereference safely, pass buffers/structs to functions, and explain null pointer risk.
[]	Pointer/array relationship	Understand practical array-to-pointer decay and pointer arithmetic basics.
[]	Bitwise operators	Use &, , ^, ~, <<, >> for masks, flags, and register fields.
[]	Bit masks	Set, clear, toggle, and test individual bits without corrupting unrelated bits.
[]	sizeof	Use safely for buffers/structs and explain common pointer-vs-array mistake.
[]	Type casting	Know when explicit casts are required and how they can hide bugs.
[]	Preprocessor conditionals	Use #if/#ifdef for configuration without turning code into unreadable spaghetti.

Memory and embedded-specific coding - SHOULD / role-track depth unless your target role touches firmware directly

Check	Skill / concept	What you must be able to prove or explain
[]	Flash/RAM concept	Explain where program code/constants and runtime data generally live on an MCU.
[]	Stack	Explain call/local-variable stack use and why very large local arrays/recursion can be dangerous.
[]	Heap	Explain dynamic allocation concept and why many embedded systems avoid malloc/free in steady-state firmware.
[]	Memory-mapped I/O	Explain that peripheral registers appear at addresses and firmware reads/writes them.
[]	Endianness concept	Explain byte order and why it matters when packing/unpacking multi-byte data.
[]	Integer overflow	Recognize wraparound and design counters/timers with bounded behavior.
[]	Blocking vs non-blocking code	Explain why long delays can make a system unresponsive; implement timer/state-machine alternatives.
[]	State machines	Implement a finite-state machine for system modes or protocol sequencing.
[]	Error codes	Return/propagate meaningful status codes and avoid silently ignoring hardware failures.
[]	Assertions/logging concept	Use debugging assertions or logs appropriately during development.
[]	Function pointers concept	Understand callbacks at a basic level; use one if a library requires it.
[]	Concurrency awareness	Know that shared state can be corrupted by interrupts/tasks and requires deliberate protection.

C proof exercises - state machine + debugging are CORE; remaining exercises strengthen firmware/embedded-test roles

Check	Skill / concept	What you must be able to prove or explain
[]	Register-mask exercise	Given an 8/16/32-bit register definition, set/clear a field correctly using masks and shifts.
[]	Ring buffer	Implement or explain a simple circular buffer for serial receive data.
[]	Packet parser	Parse a small binary or text packet with length/check fields without overflowing buffers.
[]	State machine	Write a non-blocking three-or-more-state control loop with timeout handling.
[]	Driver wrapper	Create a small reusable module around a sensor or peripheral API.
[]	Debugging	Use compiler warnings and a debugger to find at least one pointer/index/state bug; document what caused it.
[]	Warnings	Compile with strong warnings enabled and treat unexplained warnings as defects.
[]	Code review readiness	Walk another person through one of your modules and justify naming, modularity, error handling, and timing choices.

4. Python for Hardware Test & Validation

This is a major leverage point for your transition. You already have professional Python/data experience; the goal is to redirect it toward hardware validation and test automation.

Python hardware-test skills

Check	Skill / concept	What you must be able to prove or explain
☐	Serial communication	Open a COM/tty port, configure baud rate, send commands, read responses, and handle timeouts.
☐	Bytes vs strings	Correctly encode/decode ASCII and handle raw binary data.
☐	Hex/binary parsing	Convert bytes to integers/hex and parse packed fields.
☐	Structured logging	Record timestamp, test name, input, expected result, actual result, PASS/FAIL, and diagnostic details.
☐	CSV/JSON output	Save test results in a format another engineer can inspect or analyze.
☐	Exception handling	Fail tests cleanly; distinguish communication timeout, parse error, DUT failure, and test-script failure.
☐	Reusable functions/classes	Avoid one giant script; separate transport, DUT commands, tests, and reporting.
☐	Configuration file	Move ports, thresholds, addresses, or test settings out of hard-coded logic.
☐	Command-line arguments	Run tests with options such as port/device/test suite.
☐	Automated pass/fail criteria	Translate a requirement into a numeric/logic check rather than eyeballing output.
☐	Retry policy	Use retries only when justified and record them; do not hide intermittent failures.
☐	Unit testing basics	Test at least part of your parser/calculation logic with pytest or equivalent.
☐	Data visualization	Create a simple time-series/measurement plot when it helps diagnose behavior.
☐	Test summary	Produce a short machine-generated summary of passed/failed tests and key measurements.
☐	Git	Version-control Python test code next to the firmware or in a clearly linked repository.

Hardware validation thinking

Check	Skill / concept	What you must be able to prove or explain
☐	Requirement → test	Take a requirement such as "sensor update <= 100 ms" and define a repeatable test.
☐	Expected vs actual	State expected behavior before running the test.
☐	Repeatability	Run the same test multiple times and compare variation.
☐	Boundary testing	Test around minimum/maximum/threshold values instead of only nominal conditions.

Check	Skill / concept	What you must be able to prove or explain
[]	Fault injection	Disconnect a sensor, corrupt input, force timeout, or simulate invalid data and verify safe behavior.
[]	Regression testing	Re-run a known test set after firmware changes.
[]	Test independence	Know whether a failure came from the DUT, fixture, cable, power, firmware, or Python script.
[]	Evidence retention	Save logs, waveform captures, versions/commit hashes, and test conditions for failures.

5. Microcontrollers & Embedded Architecture - High-Value Bridge Preparation

MCU foundations - CORE for your portfolio; deeper register/architecture depth can grow while applying

Check	Skill / concept	What you must be able to prove or explain
[]	MCU vs MPU	Explain a microcontroller vs application processor at a practical level.
[]	ARM Cortex-M / STM32 class device	Know the family you are using, core, voltage, flash/RAM sizes, clock, major peripherals.
[]	GPIO input/output	Configure pins, read inputs, drive outputs, and respect voltage/current limitations.
[]	Pin multiplexing	Explain alternate functions and why one pin may serve UART/SPI/timer/etc.
[]	Clock tree concept	Explain system/peripheral clocks and diagnose a baud/timer problem caused by clock configuration.
[]	Reset	Explain power-on reset, reset pin, software reset, and common reset-loop symptoms.
[]	Boot modes	Understand basic boot-from-flash vs ROM/system bootloader concept.
[]	Flash programming	Build and flash firmware reproducibly.
[]	SWD/JTAG debugging	Set breakpoints, single-step, inspect variables/registers, and recover from a basic fault.
[]	Datasheet/reference-manual navigation	Locate peripheral, clock, pin, electrical, and memory information without a tutorial.
[]	HAL vs register-level	Explain tradeoff: vendor HAL accelerates development; registers give lower-level control/understanding.
[]	Memory map	Explain flash, SRAM, peripherals, and memory-mapped register regions at a basic level.

Core peripherals - timer/interrupt + GPIO are CORE; ADC/PWM/watchdog depth depends on project/role

Check	Skill / concept	What you must be able to prove or explain
[]	External interrupt	Trigger on a button/signal edge and keep ISR short.
[]	Timer	Configure a periodic event without blocking delays.
[]	PWM	Generate and measure PWM; change duty cycle programmatically.
[]	ADC	Sample an analog signal, convert raw count to voltage/engineering units, and reason about noise.
[]	Watchdog	Explain purpose and demonstrate a watchdog recovery or at least a controlled test.
[]	RTC/timestamps concept	Maintain time or elapsed-time information as needed for logging.
[]	Nonvolatile configuration	Store/read a small setting in flash/EEPROM-style storage, or explain safe approach.

Check	Skill / concept	What you must be able to prove or explain
[]	DMA concept	Explain why DMA reduces CPU involvement and implement at least one DMA-backed transfer as a SHOULD item.
[]	Low-power modes	Understand sleep/wake concept; optional unless a target role is battery/IoT heavy.

Firmware architecture - basic state-machine/timeouts/error handling are CORE

Check	Skill / concept	What you must be able to prove or explain
[]	Initialization sequence	Create a clear order: clock → GPIO → peripherals → device checks → application start.
[]	Main superloop	Write a clean basic bare-metal loop without uncontrolled blocking delays.
[]	State machine	Use explicit states and transitions for system behavior.
[]	Timeouts	Never wait forever on a peripheral; implement timeouts and error paths.
[]	Fault handling	Define safe behavior for sensor loss, bus failure, invalid command, or watchdog reset.
[]	Logging/debug output	Provide useful runtime diagnostic messages without overwhelming timing-critical code.
[]	Configuration separation	Keep pin/protocol/threshold configuration separate from application behavior when practical.
[]	Version identification	Expose firmware version/build or Git commit in logs or command interface.
[]	Bootloader/OTA concept	Explain purpose, risks, rollback concept, and integrity checking at a high level. [SHOULD]

RTOS readiness - STRETCH for Bridge Job #1; NOT an active-application blocker

Check	Skill / concept	What you must be able to prove or explain
[]	Why an RTOS	Explain when a superloop is sufficient and when scheduling/tasks improve structure.
[]	FreeRTOS tasks	Create multiple tasks with appropriate priorities and delays.
[]	Queues	Pass data safely between tasks.
[]	Semaphores	Signal events/resources correctly.
[]	Mutexes	Protect shared resources and explain priority inversion concept at a high level.
[]	Race condition	Explain and demonstrate how unsynchronized access can corrupt behavior.
[]	Deadlock concept	Explain how circular lock dependencies can freeze a system.
[]	Task timing	Measure/verify that a periodic task meets its intended period.
[]	ISR-to-task handoff	Use a safe notification/queue/semaphore pattern instead of doing heavy work in the ISR.
[]	Stack sizing awareness	Recognize task stack overflow risk and use available diagnostics. [SHOULD]

6. Communication Protocols

For the common Bridge Job #1 core, UART should be hands-on and Project 1 should use at least ONE of I2C or SPI. Both I2C and SPI become hands-on requirements for hardware-validation/embedded-test roles or when a target posting calls for them. CAN is high-value for automotive, robotics, industrial, energy, avionics, and HIL work. RS-485/Ethernet are strong role-driven additions.

UART - CORE MUST

Check	Skill / concept	What you must be able to prove or explain
☐	Frame concept	Explain baud rate, data bits, optional parity, stop bits, and asynchronous communication.
☐	TX/RX/GND wiring	Connect correctly and know that logic voltage levels matter.
☐	Baud mismatch	Recognize garbled output and verify configuration.
☐	Terminal test	Send/receive commands with a serial terminal.
☐	Firmware receive	Handle incoming characters/packets without blocking the whole system.
☐	Python integration	Send commands and parse DUT responses from Python.
☐	Logic-analyzer proof	Capture UART and verify decoded data/ baud rate.

I2C - MUST for Project 1 only if chosen as the project bus; MUST for relevant validation/embedded-test roles; otherwise SHOULD

Check	Skill / concept	What you must be able to prove or explain
☐	SDA/SCL	Explain two-wire bus, clock/data roles, shared bus, addresses.
☐	Pull-ups	Explain why they are required and recognize a missing/incorrect pull-up symptom.
☐	Addressing	Find device address in datasheet and scan/verify on the bus.
☐	Register read/write	Read/write a sensor register based on datasheet documentation.
☐	ACK/NACK	Explain acknowledgment and what NACK may indicate.
☐	Bus stuck low	Describe a recovery/debug approach.
☐	Logic-analyzer proof	Capture start/address/data/ACK/stop and interpret a transaction.

SPI - MUST for Project 1 only if chosen instead of I2C; MUST for relevant validation roles; otherwise SHOULD

Check	Skill / concept	What you must be able to prove or explain
☐	Signals	Explain SCLK, MOSI, MISO, chip select.
☐	Master/slave roles	Explain who generates clock and how chip select identifies a peripheral.
☐	Mode	Understand clock polarity/phase and recognize a mode mismatch.
☐	Speed	Know bus clock must satisfy peripheral limits.
☐	Register transaction	Implement a basic read/write to an SPI device or loopback.

Check	Skill / concept	What you must be able to prove or explain
[]	Multiple devices	Explain separate chip select lines and shared bus concept.
[]	Logic-analyzer proof	Capture/interpret at least one SPI transaction.

CAN - SHOULD / high-value; ROLE MUST for CAN-heavy automotive, robotics, industrial, energy, avionics, or HIL jobs

Check	Skill / concept	What you must be able to prove or explain
[]	Differential bus concept	Explain CAN_H/CAN_L and why differential signaling helps noise immunity.
[]	Termination	Explain 120-ohm termination concept and common missing-termination symptoms.
[]	Identifier/frame concept	Explain message IDs and arbitration at a high level.
[]	Bit rate	Configure two nodes at matching bit rate.
[]	Transmit/receive	Send and receive a CAN message between two nodes or MCU + adapter.
[]	Error handling concept	Recognize that CAN has built-in error detection and node error states.
[]	Logging	Capture CAN traffic with an adapter/tool and correlate it to system behavior.

Additional interfaces - SHOULD

Check	Skill / concept	What you must be able to prove or explain
[]	RS-232 vs TTL UART	Know they are not electrically interchangeable.
[]	RS-485	Explain differential multi-drop concept, transceiver need, termination/biasing basics.
[]	Ethernet basics	Know link, MAC/IP concept, DHCP/static IP, TCP vs UDP at a practical troubleshooting level.
[]	USB concept	Understand host/device roles and that USB is substantially more complex than UART/SPI/I2C.
[]	BLE/Wi-Fi concept	Understand modules/stacks/interfaces at a basic level; hands-on only if a project/target job benefits.

7. Lab Equipment & Hands-on Electronics

Digital multimeter - MUST

Check	Skill / concept	What you must be able to prove or explain
☐	DC voltage	Measure rail and signal voltages safely.
☐	Resistance	Measure unpowered circuits/components and interpret unexpected readings.
☐	Continuity	Check cable/trace/connection continuity and understand false confidence from a simple beep.
☐	Current	Explain correct series connection and fused-input risk; measure only when appropriate.
☐	Diode test	Check basic diode/LED junction behavior.
☐	Ground reference	Choose the correct reference point for measurements.
☐	Safety	Never place meter in current mode across a voltage source; verify leads/jacks/mode before measuring.

Bench power supply - MUST

Check	Skill / concept	What you must be able to prove or explain
☐	Voltage setpoint	Set correct voltage before connecting the DUT.
☐	Current limit	Set a conservative current limit for first power-up.
☐	Current draw	Observe quiescent/active current and identify suspicious excess current.
☐	Power-up sequencing	Bring a new board up deliberately instead of immediately applying unrestricted power.
☐	Fault response	Shut down quickly on excessive current/heat/smell and investigate before retrying.

Oscilloscope - MUST

Check	Skill / concept	What you must be able to prove or explain
☐	Probe setup	Use x10 probe, compensation check, proper ground connection.
☐	Voltage/time scales	Set volts/div and time/div to see the signal clearly.
☐	Trigger	Create a stable display using edge trigger.
☐	Frequency/period	Measure clock/PWM/serial-related timing.
☐	Amplitude/duty cycle	Measure PWM and digital/analog signal levels.
☐	Coupling concept	Know DC vs AC coupling at a basic level.
☐	Noise/ripple	Observe supply ripple or sensor noise and avoid confusing probe artifacts with real problems.
☐	Capture evidence	Save a useful waveform screenshot with notes about channel, scale, and test condition.

Check	Skill / concept	What you must be able to prove or explain
[]	Earth-referenced ground / safe probing	Understand that a typical bench-scope ground clip is earth-referenced; do not attach it to an arbitrary energized node. Know when differential/isolated methods are required. Keep personal practice to safe low-voltage electronics.

Logic analyzer - PROJECT MUST for Project 1; job-level depth is role-driven

Check	Skill / concept	What you must be able to prove or explain
[]	Digital capture	Connect safely to logic signals with common ground.
[]	Sample rate	Choose a sample rate high enough for the bus.
[]	Protocol decoder	Decode UART/I2C/SPI.
[]	Trigger/capture	Capture an intermittent or command-specific transaction.
[]	Cross-check firmware	Compare decoded bus activity to expected commands/data.
[]	Export evidence	Save capture/screenshots for project documentation and root-cause notes.

SWD/JTAG debugger - MUST for your MCU project; role importance varies

Check	Skill / concept	What you must be able to prove or explain
[]	Connect/flash	Program firmware through a hardware debugger.
[]	Breakpoints	Pause at a function/line and inspect execution.
[]	Variables	Watch variables and peripheral state.
[]	Registers	Inspect CPU/peripheral registers when needed.
[]	Call stack	Use stack/call context to understand where a fault occurred.
[]	HardFault concept	Know how you would begin investigating an MCU fault rather than power cycling blindly.

Soldering & rework - MUST

Check	Skill / concept	What you must be able to prove or explain
[]	Through-hole soldering	Create clean joints with correct wetting and minimal excess solder.
[]	Header/connectors	Install pin headers/connectors straight and without bridges.
[]	Wire preparation	Strip/tin wires and make reliable connections.
[]	Desoldering	Remove a simple component using wick/pump without destroying pads.
[]	Basic SMD	Hand-solder common larger SMD parts such as 0805/0603 passives or SOIC/TSSOP with practice.
[]	Flux	Use flux appropriately and clean residue when needed.
[]	Inspection	Identify cold joints, bridges, insufficient solder, lifted pads, wrong orientation.
[]	ESD	Use wrist strap/mat or other controlled ESD practices.

Check	Skill / concept	What you must be able to prove or explain
[]	Rework record	Photograph before/after at least one repair/rework and document why it was needed.
[]	Wire termination / crimping	Strip wire correctly, make reliable crimp/terminal connections, verify pinout and continuity, and build simple prototype harnesses neatly enough to troubleshoot later.
[]	Soldering / bench safety	Use eye protection as appropriate, manage hot tools and fumes responsibly, keep the bench organized, and power prototypes from current-limited low-voltage supplies during learning/bring-up.

Signal generator - SHOULD

Check	Skill / concept	What you must be able to prove or explain
[]	Sine/square output	Generate a known input signal and set amplitude/frequency safely.
[]	Test stimulus	Use it to validate an analog/digital input path when appropriate.
[]	Scope verification	Verify the generated signal on the oscilloscope before feeding the DUT.

8. Troubleshooting & Root-Cause Analysis

This section may matter more in interviews than memorizing additional theory. A good technician/test candidate diagnoses instead of guessing.

General debugging method

Check	Skill / concept	What you must be able to prove or explain
[]	Reproduce the failure	State exact conditions, steps, firmware/hardware version, and whether failure is consistent/intermittent.
[]	Check assumptions	Verify power, cables, pinout, orientation, configuration, and test setup before changing code.
[]	Expected vs measured	Write down what should happen and compare with real measurements.
[]	Power first	Verify input supply, regulated rails, current draw, reset, and obvious shorts before deep firmware debugging.
[]	Divide and conquer	Separate hardware, firmware, interface, fixture, and host software portions.
[]	One change at a time	Avoid changing five variables and losing the cause/effect relationship.
[]	Known-good substitution	Use a known-good cable/device/board/firmware build when possible.
[]	Instrument the system	Use logs, GPIO debug pins, scope, logic analyzer, DMM, and debugger instead of speculation.
[]	Root cause vs symptom	Do not stop at "reflashing fixed it"; determine why the original state failed.
[]	Verification	After the fix, reproduce the original test and run a regression/boundary test.
[]	Document	Capture failure, cause, correction, verification evidence, and preventive action.

Failures you must be able to talk through

Check	Skill / concept	What you must be able to prove or explain
[]	Board will not power on	Current-limit supply; inspect orientation/shorts; check input, regulator input/output, enable pins, ground; isolate rails/loads.
[]	MCU will not program	Verify target power, SWD wiring, reset/boot state, debugger settings, clock/security/readout protection possibilities.
[]	Firmware runs then resets	Check watchdog, brownout/power rail, stack/memory corruption, hard fault, reset-cause register.
[]	UART is gibberish	Check baud/clock, voltage standard, TX/RX crossing, ground, frame settings, logic-analyzer decode.
[]	I2C device not found	Check power/address/pull-ups/wiring, signal levels, bus stuck state, ACK/NACK, clock speed, datasheet timing.
[]	SPI data wrong	Check mode, chip-select timing, bit order, clock speed, wiring, transaction format, logic capture.

Check	Skill / concept	What you must be able to prove or explain
[]	ADC reading noisy	Check reference/supply, grounding, source impedance, wiring, sample timing, filtering/averaging, nearby switching noise.
[]	Button double-triggers	Recognize bounce; apply debounce logic/timing.
[]	Motor/relay resets MCU	Investigate inductive transient, flyback protection, supply droop, grounding, decoupling, driver sizing.
[]	Intermittent connector/cable failure	Continuity/flex test, visual inspection, strain relief, contact quality, known-good replacement.
[]	Python test fails but DUT may be good	Separate test-script parsing/timeout/port/config errors from actual DUT failure.
[]	Works on bench but not in enclosure/system	Consider grounding, power, EMI/noise, cable length, thermal, mechanical/connectivity, interface loading.

Root-cause proof artifacts

Check	Skill / concept	What you must be able to prove or explain
[]	Debug log #1	Document a firmware bug: symptom → evidence → root cause → fix → regression.
[]	Debug log #2	Document a wiring/protocol bug with logic-analyzer or oscilloscope evidence.
[]	Debug log #3	Document a power/electronics issue with measured voltage/current evidence.
[]	Test report	Produce at least one professional-looking 1-3 page report with requirements, setup, results, failures, conclusion.
[]	Failure story	Prepare a 2-minute interview explanation for each of the three debug logs.

9. Git, Linux & Development Workflow

Git - CORE MUST at basic project level; branches/conflicts/releases are SHOULD

Check	Skill / concept	What you must be able to prove or explain
[]	Repository	Initialize/clone a repo and understand working tree vs committed history.
[]	Status/diff	Inspect what changed before committing.
[]	Commit	Make small, meaningful commits with readable messages.
[]	Branch [SHOULD]	Create/switch/merge a feature branch.
[]	Conflict concept [SHOULD]	Resolve a simple merge conflict.
[]	.gitignore	Exclude build outputs, secrets, and generated files appropriately.
[]	Tags/releases concept [SHOULD]	Mark a demo/release version.
[]	GitHub README	Explain setup, architecture, build/flash, usage, test results, and known limitations.
[]	Issue tracking [SHOULD]	Record at least several bugs/tasks as issues or a structured project log.
[]	No secrets	Never commit passwords/tokens/private keys or personal data.

Command line / development environment - CORE basics; Linux - SHOULD / ROLE MUST when the posting requires it

Check	Skill / concept	What you must be able to prove or explain
[]	Filesystem navigation	pwd, ls, cd, mkdir, cp, mv, rm with care.
[]	Text inspection	cat/less/head/tail and basic grep.
[]	Permissions [SHOULD]	Understand executable bit and basic chmod.
[]	Processes [SHOULD]	ps, kill, basic process troubleshooting.
[]	Serial devices [SHOULD]	Recognize /dev/tty* concept and permissions.
[]	Build tools	Run compiler/build commands from terminal; understand Make/CMake concept.
[]	Environment variables [SHOULD]	Understand PATH and basic shell configuration.
[]	Package installation [SHOULD]	Install development dependencies responsibly on Linux.
[]	SSH concept [SHOULD]	Connect to a Linux target/server when needed.
[]	Network troubleshooting [SHOULD]	ip/ifconfig concept, ping, basic port/connectivity checks.

Professional development habits

Check	Skill / concept	What you must be able to prove or explain
[]	Reproducible build	Another person can clone the repo and follow your instructions to build/flash.
[]	Versioning	Record firmware/test-script version used for a test.
[]	Code formatting	Use consistent formatting/naming and avoid unreadable one-line code.

Check	Skill / concept	What you must be able to prove or explain
[]	Warnings	Do not ignore compiler warnings without a documented reason.
[]	README first impression	Repo front page immediately explains what the system does and shows a photo/block diagram.
[]	Licenses / third-party code	Credit libraries/examples and do not pretend copied code is yours.

10. Engineering Documentation & Test Discipline

Documentation you must produce

Check	Skill / concept	What you must be able to prove or explain
☐	Block diagram	Show MCU, sensors, interfaces, power, host/test connection, and major data/control flow.
☐	Schematic	Create or annotate a readable schematic for your project; all connectors/power rails/signals labeled.
☐	Pin map	List MCU pins and peripheral functions used.
☐	BOM	List key components, part numbers, quantities, ratings, and purpose.
☐	Requirements	Write a concise list of what the system must do with measurable targets.
☐	Interface definition	Document protocol, baud/address/mode, packet/command format, units, and error behavior.
☐	Test plan	Define setup, tools, steps, expected results, pass/fail criteria.
☐	Test report	Record actual results, evidence, exceptions, defects, and conclusion.
☐	Known issues	State real limitations/bugs instead of hiding them.
☐	Change log	Track meaningful design/firmware changes and why they were made.
☐	Troubleshooting notes	Maintain a chronological engineering notebook or markdown log.
☐	Safety/limits	Document voltage/current limits and any precautions for the project.

Engineering communication

Check	Skill / concept	What you must be able to prove or explain
☐	90-second overview	Explain project goal, architecture, your role, and result to a non-specialist.
☐	5-minute technical walkthrough	Explain MCU, interfaces, power, firmware architecture, test approach, and main tradeoffs.
☐	Tradeoff	Discuss one real design decision and why you chose A over B.
☐	Failure	Discuss one failed approach and what evidence changed your direction.
☐	Requirement traceability	Point from a requirement to the test that proves it.
☐	Status update	Give a concise update: completed, current risk/blocker, next action.

11. Required Project 1 - STM32 Telemetry / Control System

PURPOSE: prove embedded C + MCU + sensor interface + lab equipment + debugging + Python integration. This must be your first polished, interview-ready project.

Minimum architecture

- ☐ ARM Cortex-M / STM32-class development board (recommended; another serious MCU is acceptable if you can justify it).
- ☐ **At least one sensor/peripheral using I2C OR SPI.**
- ☐ At least one digital input (button/switch) with debouncing.
- ☐ At least one controlled output (LED/buzzer; preferably an actuator via transistor/driver).
- ☐ UART command/status interface to a computer.
- ☐ Python host program for logging/commands.
- ☐ Interrupt or timer-driven behavior - not a project built entirely around delay() calls.
- ☐ Finite-state machine or clearly structured non-blocking application logic.
- ☐ Fault/timeout behavior for a missing or invalid sensor.

Firmware checklist

- ☐ **Initialize clock/GPIO/UART/timer and the chosen bus (I2C or SPI) in a documented sequence.**
- ☐ Read sensor data and convert raw values into engineering units.
- ☐ Apply threshold/alarm logic.
- ☐ Receive at least 3 UART commands (example: status, set threshold, reset alarm).
- ☐ Use a non-blocking state machine or scheduler pattern.
- ☐ Use at least one interrupt safely.
- ☐ Implement timeout/error return from sensor transactions.
- ☐ Log useful error/status messages.
- ☐ Use modular .c/.h files rather than one giant main.c.
- ☐ Compile with warnings enabled and resolve warnings.
- ☐ Expose firmware version/build in the UART output.
- ☐ Store at least one configuration setting persistently OR document why it is not needed. [SHOULD]

Python validation/logger checklist

- ☐ Connect to the device over serial.
- ☐ Request/parse device status.
- ☐ Log timestamped sensor values to CSV or JSON.
- ☐ Issue at least one configuration command and verify the device accepted it.
- ☐ Detect/report a timeout or malformed response.
- ☐ Generate at least one plot or summary from recorded data.
- ☐ Run a short automated test sequence with explicit PASS/FAIL criteria.

Lab proof checklist

- ☐ Measure 3.3 V/5 V rail with DMM and document actual value.
- ☐ Measure current draw from bench supply in at least two operating states.
- ☐ Capture PWM/clock/interrupt-debug waveform on oscilloscope.
- ☐ **Capture and decode the chosen I2C or SPI transaction on a logic analyzer.**
- ☐ Capture and decode UART on logic analyzer.
- ☐ Document one real bug found with an instrument (not merely a code typo).
- ☐ Photograph bench setup cleanly enough that an interviewer can identify major components.

Portfolio deliverables - Project 1 is NOT complete until these exist

- ☐ GitHub repository with clean commit history.
- ☐ 1-page project summary at the top of README.
- ☐ Architecture/block diagram.
- ☐ Schematic/wiring diagram.
- ☐ Build/flash instructions.
- ☐ How-to-run instructions for Python tool.
- ☐ Requirements list.
- ☐ Test plan and results.
- ☐ Oscilloscope/logic-analyzer screenshots with captions.
- ☐ At least one debugging/root-cause story.
- ☐ Photos of finished hardware.
- ☐ 60-180 second demonstration video link.
- ☐ "What I would improve in Rev B" section.

12. Role-Track Project 2 - Automated Hardware Validation / HIL

PURPOSE: directly target Hardware Validation, Embedded Test, HIL, Product Test, and automation-heavy Integration roles. This project is **REQUIRED** before actively targeting those role families at a serious level. It is a **SHOULD**, not a universal blocker, for R&D electronics / engineering-lab / general integration roles.

Project concept

Build a device-under-test (DUT) plus a Python validation harness. The DUT can be your Project 1 board upgraded, or a second MCU. The host runs repeatable tests, sends commands, collects measurements/status, checks pass/fail criteria, and generates a report. Add a second MCU or adapter if needed to simulate external signals/faults.

Minimum DUT capabilities

- ☐ UART command interface.
- ☐ I2C or SPI peripheral.
- ☐ At least one analog input or measurable sensor.
- ☐ At least one PWM/digital output.
- ☐ Intentional fault modes (sensor missing, invalid command, timeout, out-of-range input, etc.).
- ☐ Unique firmware version/build identifier.
- ☐ Diagnostic/error codes exposed to the host.
- ☐ At least one **SHOULD**-level interface: CAN, RS-485, or Ethernet.

Python test harness

- ☐ Automated setup/check that DUT is connected and reports expected firmware version.
- ☐ Test-case functions with unique names/IDs.
- ☐ Explicit expected/actual values and PASS/FAIL.
- ☐ Timeouts and exception handling.
- ☐ Structured timestamped log.
- ☐ CSV/JSON report output.
- ☐ At least 10 automated test cases.
- ☐ At least 3 boundary/negative/fault-injection tests.
- ☐ At least one repeated/stress test (example: 1,000 command cycles or 30-minute run).
- ☐ Summary showing total pass/fail and failed-test details.
- ☐ Git commit/version included in the report when practical.
- ☐ Optional pytest/Robot Framework integration. [STRETCH]

Hardware/lab validation

- ☐ Use oscilloscope for at least one timing/electrical requirement.
- ☐ Use logic analyzer to validate at least one digital interface transaction.
- ☐ Use DMM/bench supply to verify supply/current behavior.
- ☐ Create one test where a bad cable/wiring/power condition is intentionally introduced and correctly diagnosed.
- ☐ Create one test that initially fails because of a real firmware defect; fix it and show regression results.
- ☐ Capture before/after evidence for at least one bug fix.

Professional artifacts

- ☐ Test requirements matrix.
- ☐ Test setup diagram.
- ☐ Automated test code.
- ☐ Sample passing report.
- ☐ Sample failing report.
- ☐ Root-cause report for one failure.

- ☐ Regression report after fix.
- ☐ Demo video showing one automated test sequence.
- ☐ README section explaining how this resembles professional hardware validation/HIL work.

13. Strong Differentiator Project - Custom PCB

This project is a STRONG DIFFERENTIATOR, not a universal Bridge Job #1 requirement. A working custom board materially strengthens R&D electronics, hardware integration, and embedded-hardware credibility, but do not delay applications solely to finish it.

- ☐ Define board requirements: supply input, MCU, interfaces, connectors, sensor/actuator needs.
- ☐ Create schematic in KiCad (or Altium if legitimately available).
- ☐ Select MCU/major components using datasheets and availability.
- ☐ Include power regulation and decoupling.
- ☐ Include SWD/JTAG programming/debug header.
- ☐ Include UART/debug header or connector.
- ☐ Include at least one I2C/SPI device or expansion connector.
- ☐ Include protection where appropriate (reverse polarity/ESD/flyback depending on design).
- ☐ Run ERC and resolve/justify warnings.
- ☐ Layout PCB with intentional grounding, decoupling placement, trace widths, and connector orientation.
- ☐ Run DRC and resolve/justify warnings.
- ☐ Generate BOM and fabrication files.
- ☐ Order/fabricate board.
- ☐ Perform visual inspection before power.
- ☐ Check resistance/shorts before power-up.
- ☐ Power with current limit and verify rails.
- ☐ Bring MCU up with SWD/JTAG.
- ☐ Validate each interface one at a time.
- ☐ Document at least one board-level defect or Rev-B improvement.
- ☐ Create a board bring-up checklist/report.
- ☐ Publish schematic/layout renders/photos and lessons learned.

14. Interview Readiness

Do not wait until an interview is scheduled to practice these. The interview standard is bridge-job credibility: explain what you actually built/measured/debugged, reason safely, and know what you would check next. You are not expected to answer as an experienced embedded systems engineer.

Core electronics questions you should answer confidently

- ☐ What is the difference between voltage and current?
- ☐ How do you choose a resistor for an LED?
- ☐ What happens if an MCU input is left floating?
- ☐ Why does I2C need pull-up resistors?
- ☐ What does a decoupling capacitor do?
- ☐ Why can't an MCU GPIO directly drive a motor or relay?
- ☐ What is the purpose of a flyback diode?
- ☐ What is the practical difference between a BJT and a MOSFET as a switch?
- ☐ How would you verify a 3.3 V rail?
- ☐ How would you troubleshoot a board drawing excessive current?
- ☐ What is PWM and when would you use it?
- ☐ How does an ADC convert a sensor reading into a number?
- ☐ What is the difference between an open circuit and a short circuit?
- ☐ What is the difference between an absolute maximum rating and a recommended operating condition?
- ☐ How do you read a schematic to trace a signal from an MCU to a connector?

Embedded/C questions - know the items you actually used; unused protocols, RTOS, CAN, DMA, and deeper fault-debug topics are ROLE-TRACK

- ☐ Why are bitwise operations important in embedded C?
- ☐ How do you set and clear a single bit in a register?
- ☐ What is a pointer and what can go wrong with one?
- ☐ What does volatile mean and what does it NOT guarantee?
- ☐ What is the difference between stack, heap, flash, and RAM at a practical level?
- ☐ Why might malloc/free be avoided in long-running embedded firmware?
- ☐ Interrupt vs polling: when would you use each?
- ☐ What should and should not happen inside an ISR?
- ☐ What is a finite-state machine and why is it useful?
- ☐ Why are long blocking delays a problem?
- ☐ What is a watchdog?
- ☐ [ROLE-TRACK] What is DMA and why might you use it?
- ☐ How would you debug a HardFault/reset loop?
- ☐ UART vs I2C vs SPI - compare wiring, speed, topology, and typical uses.
- ☐ What causes UART gibberish?
- ☐ What causes an I2C NACK?
- ☐ What happens if SPI mode is wrong?
- ☐ [ROLE-TRACK unless target job uses CAN] What is CAN termination and why does it matter?
- ☐ [STRETCH / ROLE-TRACK] What is an RTOS task, queue, semaphore, and mutex?
- ☐ [STRETCH / ROLE-TRACK] What is a race condition?

Lab/troubleshooting questions you should answer confidently

- ☐ A new board does not power on. Walk me through your first 10 minutes.
- ☐ How do you use a bench supply current limit during bring-up?

- ☐ When do you use a DMM vs oscilloscope vs logic analyzer?
- ☐ How would you prove whether a communication failure is hardware or firmware?
- ☐ How would you debug an intermittent failure?
- ☐ How would you validate a requirement like "response within 100 ms"?
- ☐ How do you ensure your test script is not the thing causing the failure?
- ☐ What information belongs in a root-cause report?
- ☐ What would make you stop powering a prototype immediately?
- ☐ How do you safely make a current measurement with a DMM?

Project interview stories - prepare these out loud

- ☐ 90-second career-transition story: why embedded/hardware test, and why your previous experience is useful rather than unrelated.
- ☐ 2-minute Project 1 overview.
- ☐ 5-minute Project 1 deep technical walkthrough.
- ☐ [ONLY if targeting validation/HIL] 2-minute Project 2 validation/HIL overview.
- ☐ One hardware failure story.
- ☐ One firmware failure story.
- ☐ One interface/protocol failure story.
- ☐ One Python automation story.
- ☐ One example where you used data/measurements rather than guessing.
- ☐ One example of learning an unfamiliar datasheet/tool quickly.
- ☐ One teamwork/communication story from professional experience.
- ☐ One example of making a tradeoff under time constraints.
- ☐ One honest limitation in your project and how you would improve it.

15. Role-Specific Add-On Checklists

Complete the COMMON CORE and then complete ONE add-on for the bridge-job family you are actively targeting. You do NOT need every add-on. For this checklist, "substantially complete" means every UNTAGGED checkbox in your chosen role family is demonstrable, plus its ROLE PROOF requirement when one is listed. Items marked SHOULD or STRETCH may remain open while you apply. Applying to multiple families is fine, but tailor the resume and proof package to each one.

A. Hardware Validation / Hardware Test - SELECTIVE first-bridge target; strong fit once automation/test proof is mature

- ☐ Python serial/test automation is polished and demo-ready.
- ☐ Can translate requirement → test procedure → pass/fail criterion.
- ☐ Can use oscilloscope, logic analyzer, DMM, bench supply without supervision for basic work.
- ☐ UART/I2C/SPI are hands-on strengths; CAN strongly preferred.
- ☐ Can distinguish hardware, firmware, fixture, cabling, and script failures.
- ☐ Can show automated test reports and regression results.
- ☐ Can discuss root cause using evidence.
- ☐ Basic RS-485/Ethernet familiarity. [SHOULD]
- ☐ Understand calibration/measurement uncertainty concept. [SHOULD]
- ☐ **ROLE PROOF: Project 2 is complete enough to demo automated setup, repeatable tests, explicit PASS/FAIL, logs/reports, one fault-injection case, and one root-cause/regression story.**

B. R&D Electronics / Engineering Lab Technician - PRIMARY first-bridge target

- ☐ Strong schematic-reading fundamentals.
- ☐ Confident soldering through-hole and common SMD; basic rework/desoldering.
- ☐ Can assemble prototype wiring/harnesses neatly and verify continuity/pinout.
- ☐ Can power-up prototype safely and validate rails/current.
- ☐ Can troubleshoot board-level faults methodically.
- ☐ Can read datasheets and identify substitute/component-rating constraints.
- ☐ Can keep lab/build notes and follow controlled procedures.
- ☐ Custom PCB project completed or in progress. [SHOULD]
- ☐ **Basic electromechanical prototype assembly: use common hand tools, fasteners, cable management/strain relief, and simple breakout/test-fixture hardware without damaging the electronics. [SHOULD]**
- ☐ Understand workmanship/ESD practices even without including a certification in this checklist.
- ☐ **ROLE PROOF: Show one physical build/rework dossier: prototype assembly, solder/rework, wire termination/harness work, power-up measurements, and a short troubleshooting record.**

C. Hardware Integration / Test Technician - SELECTIVE first-bridge target; many higher-paid roles expect prior integration experience

- ☐ Can read wiring diagrams/schematics and connector pinouts.
- ☐ Can verify harness continuity and signal/power routing.
- ☐ Can integrate multiple subsystems one at a time rather than "plug everything in and hope."
- ☐ Can create/use an integration checklist.
- ☐ Can log configuration/firmware/hardware revisions.
- ☐ Can diagnose interface mismatch, pinout, grounding, and power sequencing problems.
- ☐ Can write concise discrepancy/defect reports.
- ☐ Comfortable with Linux/serial/network basics. [SHOULD]
- ☐ **ROLE PROOF: Show one integration exercise/dossier: connector/pinout or harness map, staged subsystem bring-up, continuity/power checks, configuration record, and one diagnosed integration fault.**

D. Embedded Test / HIL Technician - STRETCH first-bridge target; EXCELLENT second-bridge target

- ☐ Project 2 is excellent and easy to demo.
- ☐ Python automation is strong enough to modify/add tests live.
- ☐ C is strong enough to read firmware and identify likely causes.
- ☐ CAN and UART are hands-on skills; I2C/SPI are understood at board level.
- ☐ Understand stimulus → DUT → measurement → pass/fail HIL architecture.
- ☐ Can create fault injection and negative tests.
- ☐ Can correlate logs with scope/logic-analyzer captures.
- ☐ Can explain regression testing and flaky/intermittent test handling.
- ☐ Basic CI/nightly-test concept. [STRETCH]
- ☐ **ROLE PROOF: Project 2 demonstrates stimulus → DUT → measurement → PASS/FAIL, at least one injected fault, and evidence that correlates software logs with hardware measurements.**

E. PCBA Test / Electronics Test Technician - SECONDARY first-bridge target; technically relevant, but watch compensation/schedule

- ☐ Can identify common component types and polarity/orientation.
- ☐ Can use schematic + DMM to trace power and signal paths.
- ☐ Can inspect solder joints and perform basic rework.
- ☐ Can compare a failing board to a known-good board.
- ☐ Can test rails, clocks, reset, communications, and connector I/O.
- ☐ Can document component-level fault isolation.
- ☐ Understands manufacturing test fixtures/FCT/ICT concepts at a basic level.
- ☐ **ROLE PROOF: Show one board-test/troubleshooting dossier with schematic tracing, rail/clock/reset or I/O checks as applicable, a diagnosed fault, and before/after test or rework evidence.**
- ☐ Only pursue if compensation/schedule meets your transition requirements.

16. Resume / Portfolio / Application Package

Resume

Check	Skill / concept	What you must be able to prove or explain
☐	Target headline	Make the direction clear: EE student + hardware/embedded/test focus; do not pretend to already hold an engineering degree.
☐	Skills section	List only skills you can defend in an interview: C, Python, STM32, Git, UART/I2C/SPI/CAN, lab tools, soldering, etc.
☐	Project 1 bullets	2-4 quantified/technical bullets using action + technology + result/test evidence.
☐	Role-specific proof bullets	2-4 bullets for the proof artifact relevant to that resume. Use Project 2 for validation/HIL; use prototype/rework/harness/bring-up/test evidence for other tracks.
☐	Current work translation	Emphasize transferable electrical systems, technical drawings, coordination, troubleshooting; do not let BIM dominate the target resume.
☐	Prior Python/data experience	Make professional Python/automation/data capability visible because it differentiates you for validation roles.
☐	No inflated claims	Do not call tutorial exposure "proficient" and do not call personal projects professional experience.
☐	One-page target	Aim for a focused one-page resume at this stage unless a compelling reason requires two pages.
☐	Separate target versions	Have at least Hardware Test/Validation and R&D Electronics versions, with same truth but different emphasis.

GitHub / portfolio

Check	Skill / concept	What you must be able to prove or explain
☐	Pinned projects	Pin Project 1 plus the strongest role-specific proof project/artifact. Pin Project 2 only when it is relevant to the target role.
☐	Professional username/profile	Reasonable profile name/photo/bio; no distracting unfinished junk as the first impression.
☐	README quality	Each project explains problem, architecture, build, test, results, photos, and lessons.
☐	Code quality	No secrets, no huge binaries, no unexplained copied vendor examples as the majority of the project.
☐	Evidence	Waveforms, logic captures, schematics, test reports, photos, and demo links.
☐	Commit history	Shows progress rather than one giant "final project" upload.
☐	Issues/lessons	At least some evidence of defects/troubleshooting/design iteration.

Check	Skill / concept	What you must be able to prove or explain
[]	Contact	Professional contact/LinkedIn link if desired; no home address or unnecessary personal data.

Application readiness

Check	Skill / concept	What you must be able to prove or explain
[]	Target job titles	Hardware Validation Technician; Hardware Test Technician; Engineering Lab Technician; R&D Electronics / R&D Engineering Technician; Hardware Integration/Test Technician; Embedded Test/HIL Technician; Electronics Engineering Technician; Prototype Technician; Product Test Technician.
[]	Permanent/full-time filter	Reject internships/contracts/temporary roles unless an unusually safe exception is deliberately chosen.
[]	Compensation floor	Define your real minimum before interviews; evaluate total benefits, not salary alone.
[]	School schedule filter	Ask exact work hours, overtime, travel, rotating shifts, on-call, and flexibility before accepting.
[]	Engineering adjacency	Confirm the employer has electrical/firmware/hardware/R&D engineers you can learn from or move toward.
[]	Job-content filter	Prefer product development, prototypes, PCBs, validation, test automation, firmware interaction, lab work over installation/maintenance-only roles.
[]	Internal mobility	Ask how technicians have moved into engineering or test roles and whether tuition assistance exists.
[]	Interview tracker	Track job, role family, salary, skills requested, questions asked, outcome, and skill gaps revealed.
[]	No desperation move	Do not leave a stable job just because a new title contains "electronics." The move must improve the whole career/school/financial system.
[]	Hard-requirement screen	Before investing heavily in an application, check true hard barriers such as required degree/certification/license, security clearance eligibility, work authorization, shift/travel requirements, and minimum experience when stated as non-negotiable. Do not assume a portfolio overrides a legal/regulatory or explicitly hard requirement.
[]	Required vs preferred judgment	Do not self-reject because you miss preferred skills or every year-of-experience wish-list item. If there is no hard barrier and you can perform most core duties, apply and let the employer decide.

17. Final Green-Light Scorecard

FINAL USE: This page has TWO different decisions. (1) ACTIVE APPLICATIONS begin once Gates 1-7 are green and ONE target-role add-on is ready. (2) LEAVING YOUR CURRENT JOB happens only after Gate 8 is satisfied by a real offer. Do not confuse "ready to interview" with "must resign."

A. Electronics & schematics core - MUST

- ☐ Can reason through basic DC circuits, common components, digital levels, pull-ups, transistor/MOSFET switching, power rails/decoupling, and simple sensor/actuator interfaces.
- ☐ Can read schematics and datasheets to find pinouts, ratings, connector signals, operating conditions, and the information needed to troubleshoot safely.

B. Lab, fabrication & safe bring-up - MUST

- ☐ Can safely use a DMM, current-limited bench supply, and oscilloscope; can make basic voltage/resistance/continuity/current and timing/signal measurements.
- ☐ Can solder and inspect basic electronics, perform simple rework/wire termination, follow ESD practices, and power up a prototype methodically.

C. Programming & automation core - MUST

- ☐ Can write and explain basic modular C for the MCU project (functions, arrays/structs, pointers at a practical level, bitwise operations, warnings, state-machine/error-handling basics).
- ☐ Can use Python for serial communication, logging, parsing, simple automated pass/fail checks, and saving results; prior professional Python skill is translated into hardware-test use.
- ☐ Git and basic command-line workflow are usable enough to clone/build/run/document project code. Linux familiarity is SHOULD unless your target role requires it; deeper firmware architecture is not a universal Bridge Job #1 blocker.

D. Project 1 - MUST

- ☐ One polished STM32/ARM-class project is complete and demo-ready: GPIO + timer/interrupt + UART + at least ONE of I2C/SPI + Python host tool + measurement/debug evidence.
- ☐ Project documentation includes a block diagram/schematic, build instructions, test evidence, photos, Git history, and at least one genuine debugging/root-cause story.

E. Troubleshooting & engineering discipline - MUST

- ☐ Can use a reproduce → measure → isolate → change one thing → verify process and can walk through at least 3 representative failures, including one physical/electrical fault.
- ☐ Can write concise test steps, expected vs actual results, pass/fail criteria, a defect/root-cause note, and a short technical status update.

F. ONE target-role add-on - MUST

- ☐ Choose ONE Section 15 family. Every UNTAGGED checkbox in that chosen track is demonstrable, and its ROLE PROOF requirement is complete; SHOULD/STRETCH items may remain open.
- ☐ Project 2 is required only for serious Hardware Validation / Embedded Test / HIL targeting. Other role families may use a relevant prototype/rework/harness/bring-up/troubleshooting proof artifact instead.

G. Professional package - MUST

- ☐ Targeted resume(s), GitHub/portfolio, project links, 90-second transition story, and 5+ truthful technical/behavioral stories are ready.
- ☐ Can explain how prior Python/data work plus current electrical/VDC experience transfers into test, troubleshooting, documentation, and hardware-system work without overstating it.

H. High-value SHOULD / keep building while applying

- ☐ Second board-level serial bus (both I2C and SPI hands-on) and logic-analyzer depth.
- ☐ CAN hands-on when target industries benefit from it.

- [] Project 2 / automated HIL validation if not already required by your chosen target track.
- [] FreeRTOS, RS-485/Ethernet, and deeper SWD/register-level debugging as role-driven growth.
- [] Custom PCB, signal generator/spectrum-analyzer exposure, and additional fabrication depth.
- [] Basic C++ familiarity for firmware-heavy roles. None of these H items should delay applications unless a specific target posting makes one essential.

Decision

Check	Decision	Standard
[]	START ACTIVE APPLICATIONS	Gates 1-7 are green AND at least one Section 15 target-role add-on is substantially complete. Do not wait for Gate 8; that is evaluated when an offer exists.
[]	KEEP PREPARING	A core Gate 1-7 area is materially weak, unsafe, or cannot be demonstrated, OR you have not yet built proof for any target-role family.
[]	ACCEPT / LEAVE CURRENT JOB	Only after a real offer satisfies Gate 8: permanent/full-time, acceptable total compensation/benefits, school-compatible schedule, and genuine hardware/electronics/R&D relevance.

My start date for active applications:

Date: _____ Initial target role family: _____

Minimum acceptable compensation / benefits / schedule notes: _____

Appendix - Market Signals Used to Build This Checklist

Market calibration checked September 6, 2026. Re-check live postings when you begin the application campaign. These sources are examples used to calibrate the SKILL THRESHOLD, not a recommended-job list and not promises that the exact postings will remain open. Some examples are deliberately more senior than your first bridge job so the checklist can distinguish what belongs in the common core from what belongs in a later role track. R&D/engineering-lab technician roles emphasize prototyping, soldering, schematics, instruments, and sometimes Linux/Python; hardware validation roles demand more protocol/test-automation depth; integration roles emphasize wiring/harnesses, drawings, bring-up, and fault isolation. The final gate therefore uses a common core + ONE role track instead of requiring engineer-level breadth before applying.

Source	Skill signals	URL
Gradient Technologies - Junior Firmware Engineer (Alpharetta, GA)	C/C++, STM32/Nordic/ESP32, board bring-up, peripheral testing, unit tests, projects/capstone evidence, product lifecycle.	https://gradienttech.com/careers-at-gradient-technologies/
Capgemini - Verification & Validation Engineer, Avionics	Python hardware-validation scripts, requirement-to-test translation, UART/RS-422/485/SPI/I2C/GPIO/Ethernet/CAN, oscilloscopes, logic analyzers, DMMs, power supplies, signal generators, root cause, Git, traceability.	https://careers.capgemini.com/job/Atlanta%2C-GA-Verification-and-Validation-Engineer-Avionics-GA-30301/1413037433/
Specter - R&D Electronics Technician	High-complexity solder/rework, PCBA/harness work, engineering drawings/schematics, prototype support, direct collaboration with engineering.	https://jobs.ashbyhq.com/specter/d43879ce-2c07-4a98-abce-2f8edba5a8c8
Flock Safety - Engineering Lab Technician (Atlanta, GA)	PCB-level prototyping, board bring-up, soldering, wire termination, oscilloscopes/multimeters, schematics, firmware deployments, Linux/Python automation, test data, technical SOPs.	https://jobs.ashbyhq.com/flock%20safety/2d97b86c-aba4-4e88-9007-b003dd5931d4
MAG Aerospace - Hardware Integration/Test Technician (Atlanta, GA)	Cable/harness fabrication and connector termination, wiring diagrams/schematics, DMM/oscilloscope use, staged system bring-up, acceptance-test execution, fault isolation, as-built/red-line documentation, ESD/workmanship discipline.	https://careers-magaero.icims.com/jobs/8272/hardware-integration-test-technician/job?in_iframe=1

Final principle: The goal is to become demonstrably useful in an electronics/hardware environment before you ask an employer to take a chance on your career transition. Start applying once the COMMON CORE, Project 1, and ONE target-role add-on are demonstrable. Do not wait until you have the full skill set of an embedded systems engineer, do not wait for every role family, and do not wait for the associate degree solely because it feels safer.